Ebb — Requirements & System Design

Oct 15, 2026 · @Yasin

Overview

Ebb is an adaptive AI study tutor that quizzes students, grades their answers, teaches what they got wrong, and brings concepts back right before they are forgotten. It is a portfolio project built to show agent engineering with LangGraph, not just a chat wrapper.

The core idea is the Ebbinghaus forgetting curve: knowledge ebbs over time, and well-timed review raises the tide again. Every concept carries a review stage, and the agent schedules reviews from it.

Scope. A web app with sign-in, subjects built from a topic or uploaded notes, four study modes (Quiz, Flashcards, Exam, Chat), cited answers, grade disputes, exam PDF export and a progress dashboard.

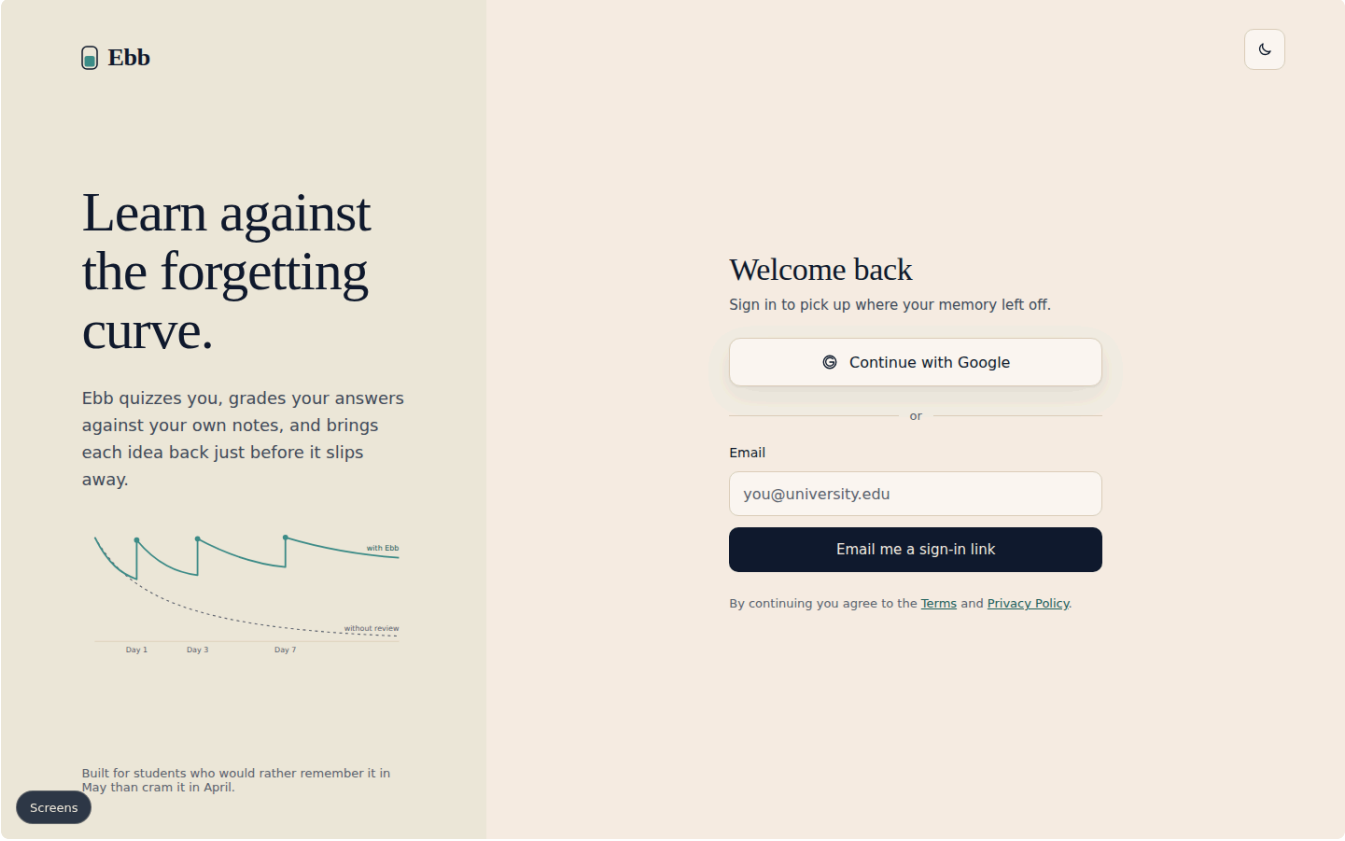
Constraints. Free tiers only for now. No local models (no GPU). Grounding stays simple: Postgres full-text search over uploaded notes, no vector database.

Success looks like: a deployed app a reviewer can use end to end, a grading eval with a measured agreement rate, and a design that can be explained on a whiteboard in five minutes.

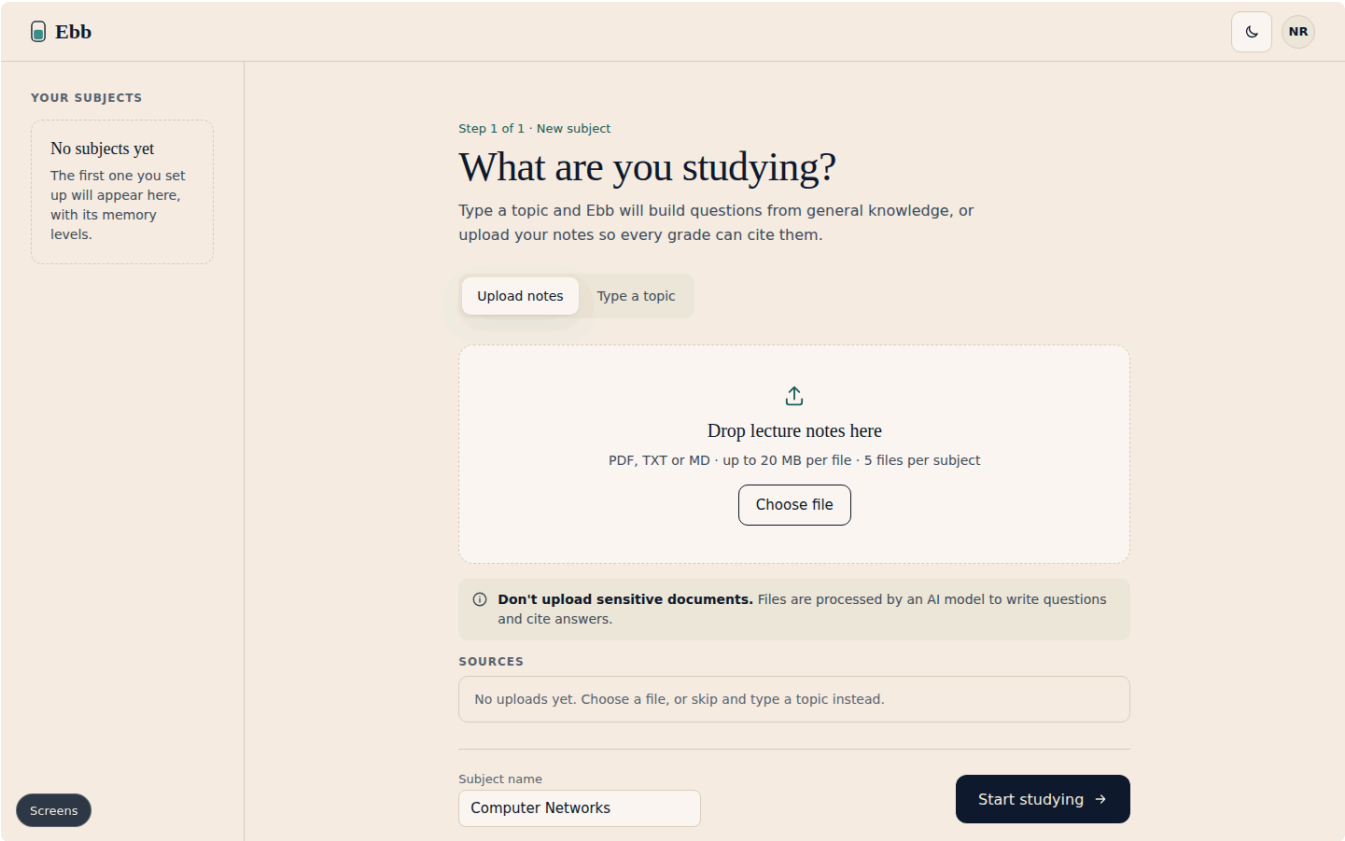
Screens

The prototype has six desktop screens plus mobile layouts, all using mock data for one learner studying Computer Networks. Screenshots were rendered headless, so headings show a fallback serif rather than Fraunces.

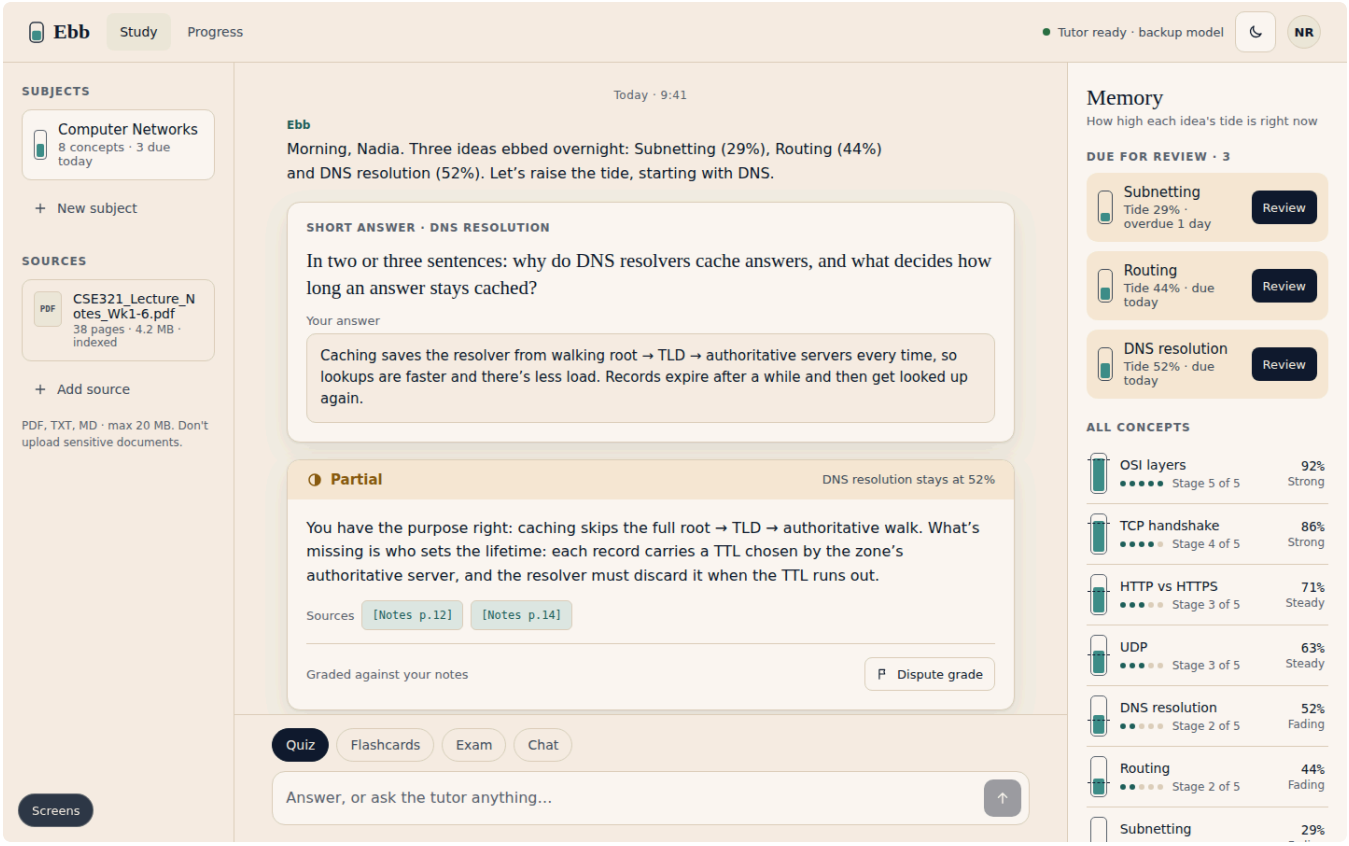
01 Sign in. Google OAuth or email (FR-14).



02 Onboarding. Start a subject from a typed topic or uploaded notes, with the upload limits and privacy notice (FR-1, NFR-9, NFR-10).



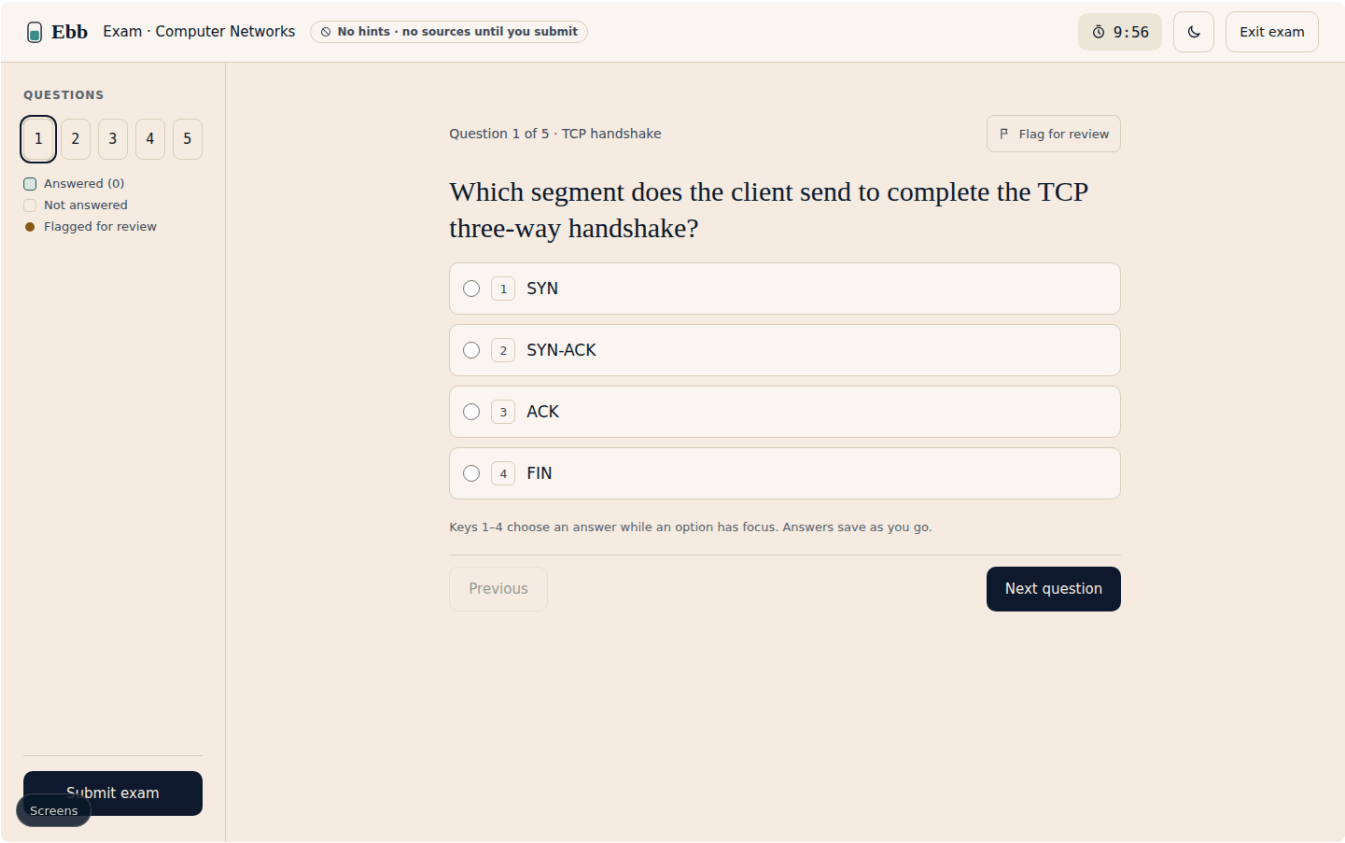
03 Study workspace. Subjects and sources on the left, the tutor chat with mode chips in the centre, and the memory panel with tide gauges and due reviews on the right. The grade card shows a Partial grade, citation chips and the Dispute grade button (FR-2 to FR-9).



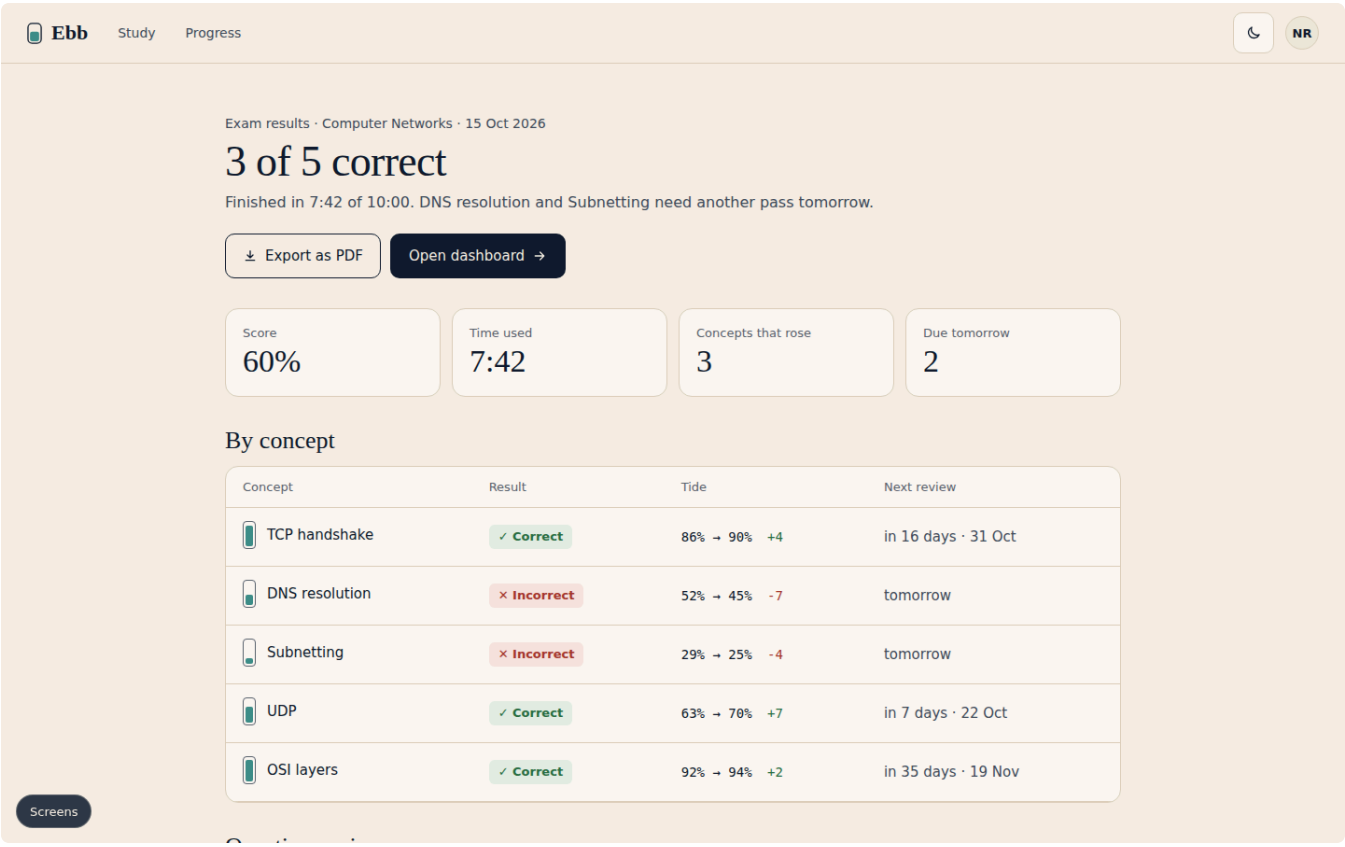
04 Flashcards. Cards from weak concepts, rated Got it or Still learning (FR-10).



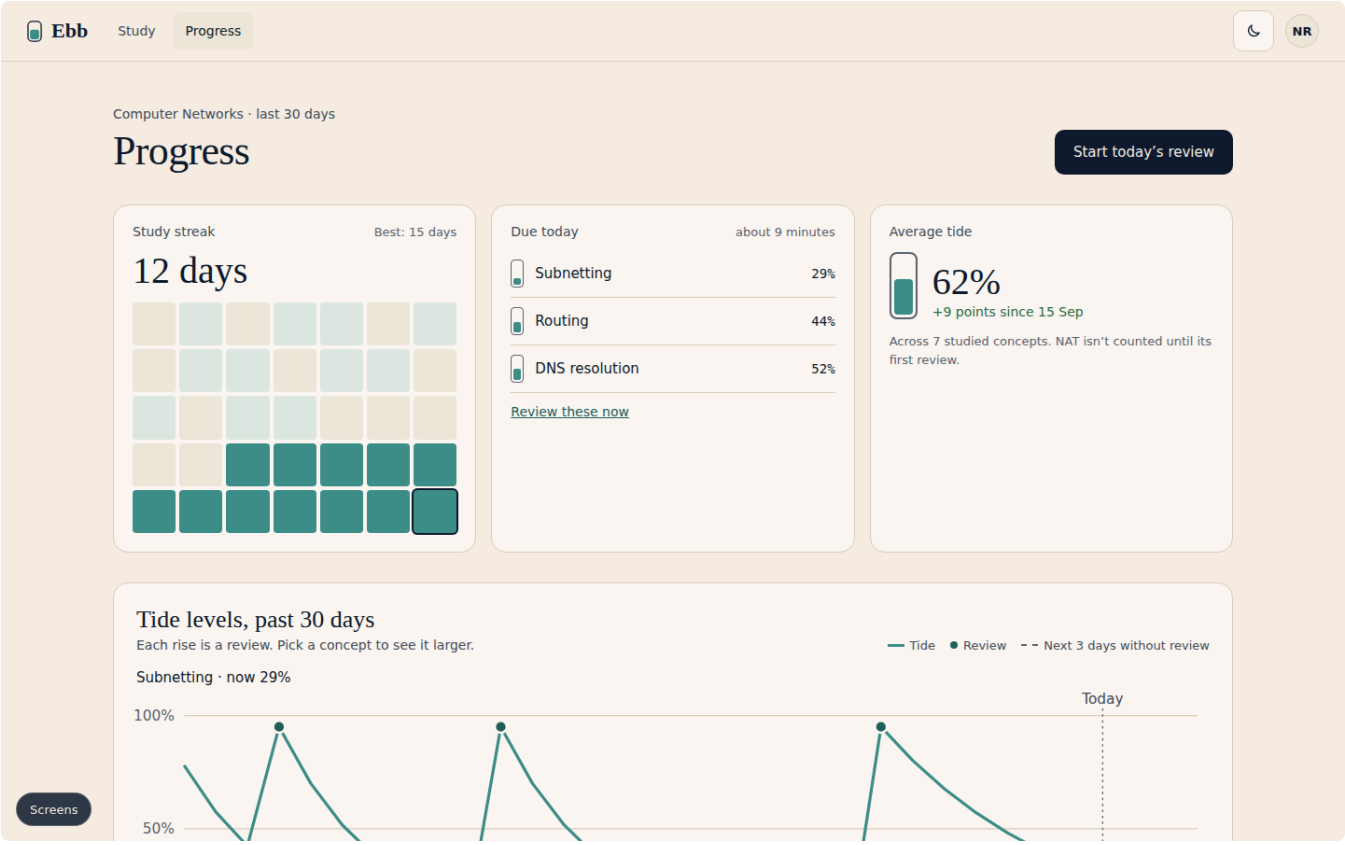
05 Exam. Timed, no hints, with a question navigator (FR-11).



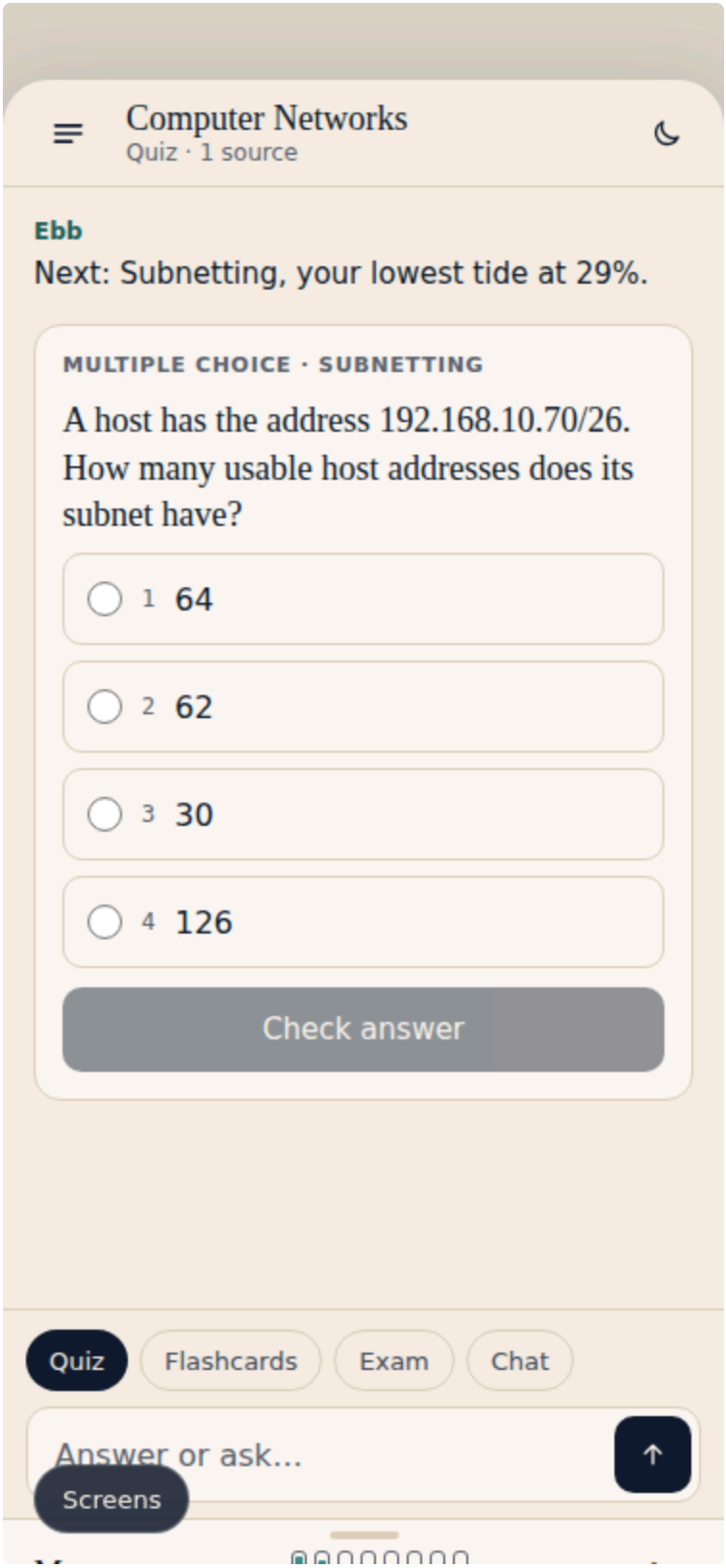
05b Exam results. Score broken down by concept, question review and PDF export (FR-11, FR-13).



06 Progress dashboard. Study streak, due today, average tide and tide levels over 30 days (FR-14).



Workspace on mobile (375 px). The memory panel becomes a bottom sheet (NFR-15).


Computer Networks
Quiz · 1 source
Ebb
Next: Subnetting, your lowest tide at 29%.
MULTIPLE CHOICE · SUBNETTING
A host has the address 192.168.10.70/26. How many usable host addresses does its subnet have?
1 64
2 62
3 30
4 126
Check answer
Quiz
Flashcards
Exam
Chat
Answer or ask...
Screens

Functional requirements

Fourteen requirements, prioritized so the MVP is the Must rows. Every decision step (scheduling, mastery, routing) is plain code; the LLM only generates and grades.

ID	Priority	Requirement	How it is solved
FR-1	Must	Create a subject from a typed topic or uploaded notes	Onboarding screen posts to FastAPI; an LLM call with structured output extracts a concept list, saved to `concepts` at stage 0
FR-2	Must	Generate questions at varying difficulty, multiple choice and short answer	`generate_questions` node produces a batch of about 5 questions as Pydantic objects, cached in graph state to save rate limit
FR-3	Must	Grade answers with an explanation and the misconception	`grade_answer` node with a reference answer and rubric in the prompt, low temperature, structured `Grade` output
FR-4	Must	Track mastery per concept	`update_mastery` applies Leitner rules in Python: correct moves up a stage, partial stays, incorrect returns to stage 1
FR-5	Must	Adapt what comes next	`route_next` conditional edge in code: reteach on a miss, harder question on a streak, next concept when done
FR-6	Must	Bring concepts back when due ("do you still remember?")	`pick_concept` queries `concepts.next_review_at <= now`; intervals 1, 3, 7, 16, 35 days
FR-7	Must	Resume a session after leaving	LangGraph `PostgresSaver` checkpointer keyed by `study_sessions.thread_id`
FR-8	Must	Teach from uploaded notes with citations	Postgres full-text search picks chunks; the LLM must cite chunk IDs; `verify_citations` rejects unknown IDs and retries at most twice

ID	Priority	Requirement	How it is solved
FR-9	Should	Dispute a grade	`interrupt` pauses the graph; `regrade` re-checks with a stricter prompt; overturned grades correct the stage and log `grade_disputes`
FR-10	Should	Flashcards from weak concepts	Flashcard subgraph generates cards for low-stage concepts; Got it / Still learning are logged as attempts and update the stage
FR-11	Should	Timed mock exam with results by concept	Exam subgraph reuses question generation and grading, skips teaching; answers graded on submit
FR-12	Should	Find helpful resources when stuck	`teach` node may call one Tavily or YouTube search; only returned URLs can be cited
FR-13	Should	Export an exam paper as PDF	WeasyPrint renders an HTML template of the stored exam; file saved to Supabase Storage, path in `exams.pdf_path`
FR-14	Should	Sign in and see a progress dashboard	Supabase Auth with Google; dashboard reads stages, due counts and streak from `concepts` and `attempts`

Non-functional requirements

Targets are starting points to adjust once measured. Free-tier limits shape most of them.

ID	Quality	Target	How it is solved
NFR-1	Responsiveness	First token in about 2 s or less	Token streaming over Server-Sent Events using LangGraph's `messages` stream mode; the UI shows a streaming state

ID	Quality	Target	How it is solved
NFR-2	Grading accuracy	At least 90% agreement with hand labels	A golden set of 30 to 50 labeled answers, rerun in LangSmith after every prompt change
NFR-3	Grounding	Zero invented citations	Citation IDs checked in code; quoted snippets checked against source text; unsourced answers labeled "General knowledge"
NFR-4	Reliability	No crash on malformed LLM output	Pydantic validation, `.with_retry()`, bounded retry counters in graph state
NFR-5	Availability under rate limits	Graceful switch, no failed turn	`.with_fallbacks()` from Gemini to Groq; the UI shows a subtle backup-model notice
NFR-6	Cost	Stay within free tiers	About 3 LLM calls per question cycle; questions generated in batches; all decision logic in code
NFR-7	Security	One user can never read another's data	Supabase JWT verified in FastAPI; row-level security with `user_id = auth.uid()` on every table
NFR-8	Prompt-injection safety	Uploaded notes cannot change agent behavior	Retrieved text wrapped and labeled as data in prompts; grades and citations validated in code
NFR-9	Privacy	Users know what not to upload	Upload notice; free API tiers may use prompts to improve products
NFR-10	Upload limits	20 MB per file, 5 files per subject, PDF, TXT, MD	Checked in the FastAPI upload endpoint; parsing runs off the event loop
NFR-11	Observability	Every routing decision traceable	LangSmith tracing; `attempts.model_used` records which model graded

ID	Quality	Target	How it is solved
NFR-12	Maintainability	Core logic covered by tests	Nodes as small pure functions; prompts in their own files; pytest for Leitner rules and the citation verifier
NFR-13	Deployability	One command to run locally	`docker compose up`; secrets from environment variables via `pydantic-settings`
NFR-14	Cold starts	No frozen screen when the free host sleeps	"Waking up the tutor" state in the UI; health-check ping before the first request
NFR-15	Usability and accessibility	WCAG AA, works at 375 px	shadcn/ui on Radix primitives; keyboard answering (keys 1 to 4); memory panel becomes a bottom sheet on mobile

Tech stack

Python for the agent and API, TypeScript for the frontend, and free tiers throughout. Free-tier terms change often, so limits should be rechecked before relying on them.

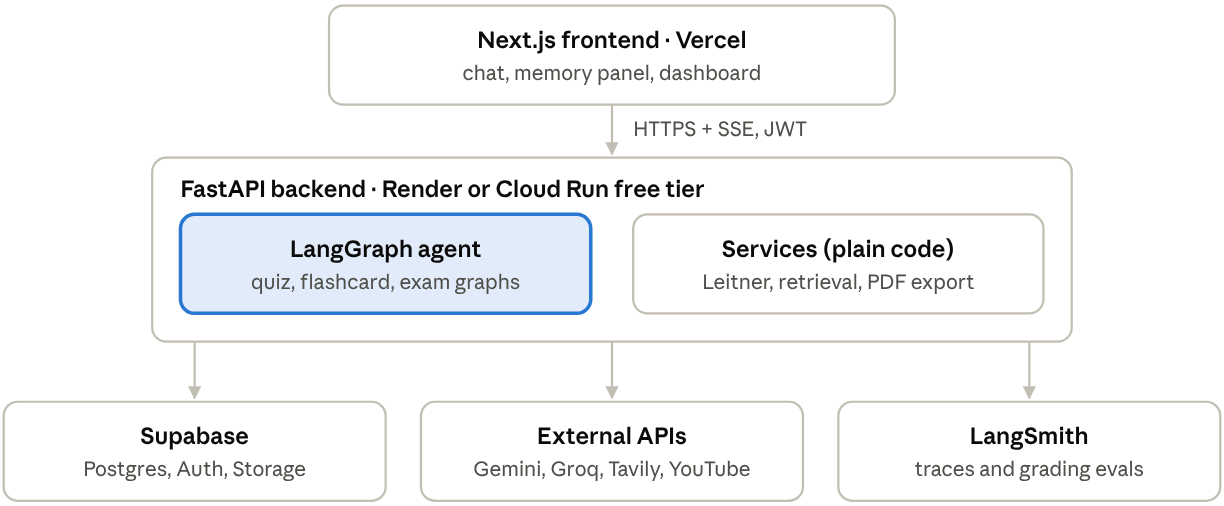
Layer	Choice	Why
Agent framework	LangGraph + LangChain	Explicit state, conditional routing, checkpointing and `interrupt` for human-in-the-loop
Primary LLM	Gemini API free tier (Flash models)	Strong grading quality and a large context window
Fallback LLM	Groq free tier	Very fast; no GPU locally, so this is the final backup
Web search	Tavily free tier	Built for agents, integrates with LangChain
Video links	YouTube Data API	Free daily quota

Layer	Choice	Why
Retrieval	Postgres full-text search (`tsvector` , GIN index, `ts_rank`)	No in-memory index to rebuild after a free host sleeps; one SQL query
PDF text extraction	`pypdf`	Free and permissive license
Backend API	FastAPI + `pydantic-settings`	Async, Pydantic-native, streaming responses
Database	Supabase Postgres	Also stores LangGraph checkpoints; row-level security built in
Auth	Supabase Auth (Google OAuth)	JWT the backend can verify; no password handling of our own
File storage	Supabase Storage	Uploaded notes and exported exam PDFs
Frontend	Next.js (App Router) + TypeScript	Matches the target CV stack
UI components	shadcn/ui (Radix + Tailwind), lucide icons, Recharts	Accessible primitives; we own the component code, including the tide gauge
Fonts	Fraunces (headings), Inter (UI)	Free on Google Fonts
PDF export	WeasyPrint	HTML template to PDF
Tracing and evals	LangSmith free developer tier	Traces every run; datasets for the grading eval
Tooling	`uv` , pytest, Docker Compose	Fast environments, tests for core logic, one-command local run
Hosting	Vercel Hobby (frontend), Render or Cloud Run free tier (backend)	Free; backend may sleep when idle (NFR-14)

Architecture and agent graph

The browser only talks to FastAPI, and the LangGraph agent runs inside the backend. Learning state lives in our own tables; the checkpointer only holds the current session.

The browser talks only to FastAPI; the agent runs inside it

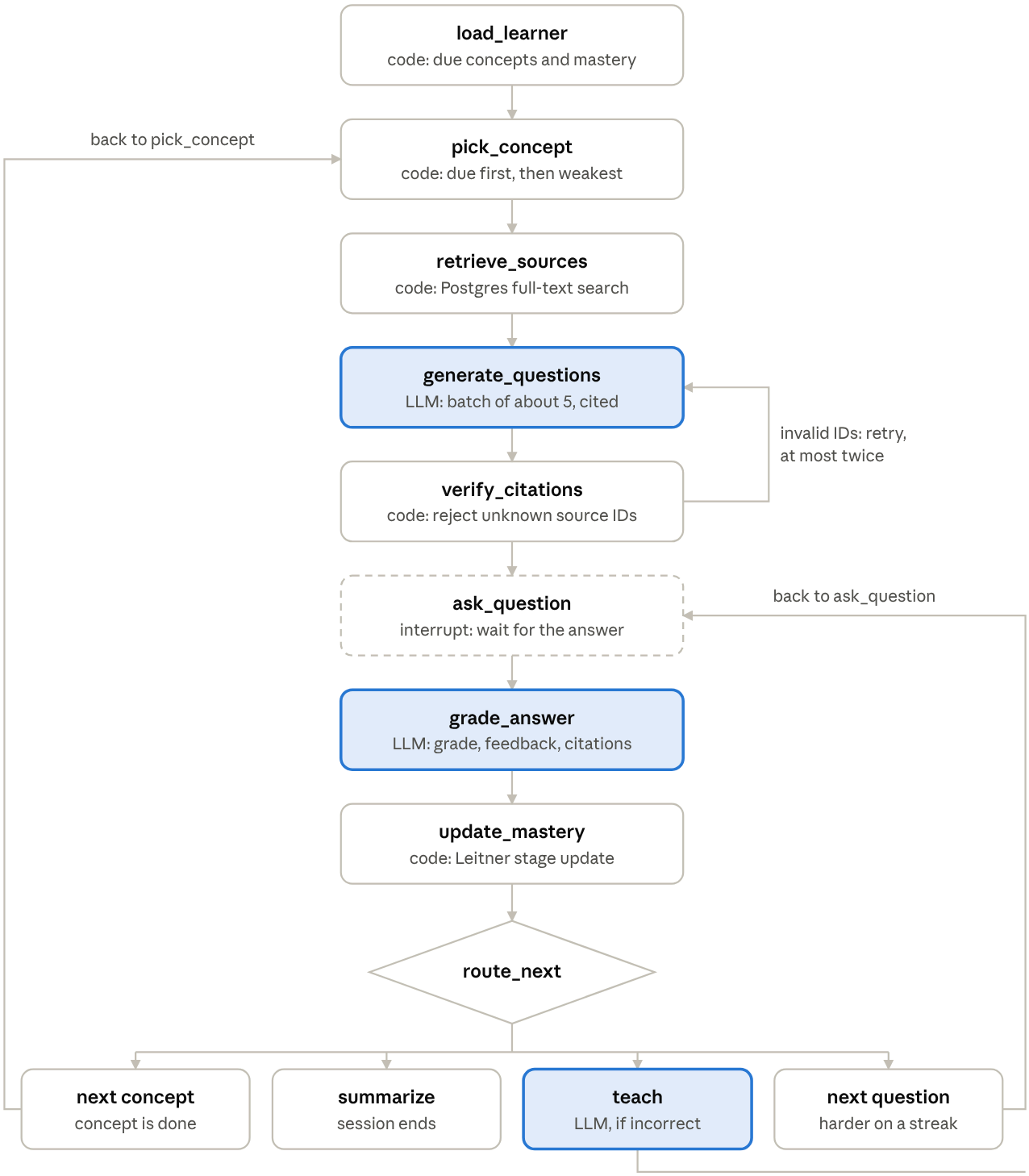


system architecture · free tiers

FastAPI verifies the Supabase JWT on every request and streams tokens back over Server-Sent Events.

Three LLM calls per question; code makes every decision

Tinted box = LLM call · dashed box = pauses for the user · plain box = code



quiz loop · 9 nodes, 4 outcomes

Only `generate_questions`, `grade_answer` and `teach` call an LLM. Flashcard and exam modes are subgraphs that reuse these nodes, and a grade dispute enters through an interrupt and a `regrade` node.

Database schema

Twelve tables in Supabase Postgres, with `attempts` as the log that powers the dashboard, streaks and evals. Supabase provides `auth.users`; LangGraph manages its own checkpoint tables, linked only through `study_sessions.thread_id`.

```
erDiagram
  PROFILES ||--o{ SUBJECTS : owns
  SUBJECTS ||--o{ DOCUMENTS : has
  DOCUMENTS ||--o{ DOCUMENT_CHUNKS : "split into"
  SUBJECTS ||--o{ CONCEPTS : contains
  SUBJECTS ||--o{ STUDY_SESSIONS : "studied in"
  STUDY_SESSIONS ||--o{ QUESTIONS : generates
  STUDY_SESSIONS ||--o| EXAMS : "may be"
  CONCEPTS ||--o{ QUESTIONS : tests
  CONCEPTS ||--o{ FLASHCARDS : "summarized as"
  QUESTIONS ||--o{ ATTEMPTS : "answered in"
  FLASHCARDS ||--o{ ATTEMPTS : "reviewed in"
  CONCEPTS ||--o{ ATTEMPTS : "updated by"
  ATTEMPTS ||--o| GRADE_DISPUTES : "may have"
  EXAMS ||--o{ EXAM_QUESTIONS : includes
  QUESTIONS ||--o{ EXAM_QUESTIONS : "appears in"
```

Table	Key columns	Purpose
`profiles`	`id` (= `auth.users.id`), `display_name`, `timezone`	User settings; timezone decides "due today" and streak days
`subjects`	`id`, `user_id`, `name`	One per course a learner studies
`documents`	`id`, `subject_id`, `filename`, `storage_path`, `size_bytes`, `page_count`, `status`	Uploaded notes; file bytes live in Supabase Storage

Table	Key columns	Purpose
`document_chunks`	`id`, `document_id`, `subject_id`, `page_start`, `page_end`, `heading`, `content`, `search_vector`	Retrieval units; `search_vector` is a generated `tsvector` with a GIN index
`concepts`	`id`, `subject_id`, `name`, `stage` (0 to 5), `next_review_at`, `last_reviewed_at`, `review_count`	Concepts plus their mastery; tide level is computed, not stored
`study_sessions`	`id`, `subject_id`, `mode`, `thread_id`, `started_at`, `ended_at`	Links a study session to its checkpointer thread
`questions`	`id`, `session_id`, `concept_id`, `type`, `difficulty`, `prompt`, `options`, `reference_answer`, `citations`, `is_general_knowledge`	Generated questions with verified citations
`flashcards`	`id`, `concept_id`, `front`, `back`, `citations`	Cards for weak concepts
`attempts`	`id`, `question_id` or `flashcard_id`, `concept_id`, `exam_id`, `answer_text`, `grade`, `feedback`, `stage_before`, `stage_after`, `model_used`, `graded_at`	Every answer from quiz, flashcards and exams
`grade_disputes`	`id`, `attempt_id`, `reason`, `original_grade`, `status`, `resolved_at`	Dispute history; overturned ones become eval cases
`exams`	`id`, `session_id`, `duration_seconds`, `started_at`, `submitted_at`, `score`, `pdf_path`	Timed exams and their exported PDF
`exam_questions`	`exam_id`, `question_id`, `position`	Question order within an exam

Rules that hold across the schema:

- **Security:** every table carries `user_id`, so one row-level security policy (`user_id = auth.uid()`) protects all of them.

- **Indexes:** `concepts (user_id, next_review_at)` for due reviews; GIN on `document_chunks.search_vector`; `attempts (user_id, graded_at)` for the dashboard and streak.
- **Integrity:** a check constraint on `attempts` requires exactly one of `question_id` or `flashcard_id`; stages are constrained to 0 to 5.
- **Citations:** stored as `jsonb` lists of `{type: chunk or web, id, label}` and verified in code before any row is written.

Decisions and build milestones

Decision	Reason
Gemini primary, Groq final fallback, no local models	No GPU; free tiers only
Users pick the mode with chips; no LLM intent router	Saves an LLM call per turn and stays deterministic
Learning state in our own tables, not the checkpointer	The checkpointer is per thread and hard to query for a dashboard
All decision logic in code; LLMs only generate and grade	Cheaper, testable, explainable
Citations verified in code with a bounded retry	Prompts alone cannot stop invented sources
Questions generated in batches of about 5	Biggest rate-limit saver
Postgres full-text search instead of `rank_bm25`	Persistent index survives free-host sleep; one fewer library
Flashcard self-ratings update the stage	Classic Leitner behavior
Review intervals 1, 3, 7, 16, 35 days	Taken from the prototype
`user_id` on every table	One row-level security rule protects everything

Build order, brain before face:

1. Core loop in the terminal: topic, question, grade, mastery update, routing (FR-1 to FR-5).

2. Persistence and spaced review (FR-6, FR-7).
3. Uploads with cited answers, grade disputes, and the grading eval (FR-8, FR-9, NFR-2).
4. FastAPI backend and the Next.js frontend from the prototype, including the memory panel and dashboard.
5. Flashcards, exam mode, PDF export, resource search (FR-10 to FR-13).
6. Auth and deployment (FR-14, NFR-13, NFR-14).